

**Bases conceptuales acerca del lenguaje unificado
de modelado (UML) y patrones de diseño**

GA4-220501095-AA2-EV03

APRENDIZ:

Johan Mauricio0 Sánchez Gaviria

Ficha: 3134555

Servicio Nacional de Aprendizaje SENA

Tecnólogo en Análisis y Desarrollo de Software

Tabla de contenido

1. Introducción.....	3
2. Taller de bases conceptuales acerca del lenguaje unificado de modelado (UML) y patrones de diseño.....	4
3. Glosario	6
4. Conclusiones	7
5. Bibliografía	8

1. Introducción

Los lenguajes orientados a objetos dominan el mundo de la programación porque modelan los objetos del mundo real. UML es una combinación de varias notaciones orientadas a objetos: diseño orientado a objetos, técnica de modelado de objetos e ingeniería de software orientada a objetos.

UML usa las fortalezas de estos tres enfoques para presentar una metodología más uniforme que sea más sencilla de usar. UML representa buenas prácticas para la construcción y documentación de diferentes aspectos del modelado de sistemas de software y de negocios.

2. Taller de bases conceptuales acerca del lenguaje unificado de modelado (UML) y patrones de diseño

El Lenguaje Unificado de Modelado (UML) es un estándar para visualizar, especificar, construir y documentar los artefactos de software. UML proporciona una forma de ver el diseño de un sistema desde diferentes perspectivas a través de varios tipos de diagramas. Estos diagramas ayudan a los desarrolladores y otras partes interesadas a entender, discutir y documentar la arquitectura y el comportamiento del sistema.

UML se utiliza principalmente en el desarrollo de software orientado a objetos, aunque también puede aplicarse a otros paradigmas. Fue creado en la década de 1990 por Grady Booch, Ivar Jacobson y James Rumbaugh, quienes unificaron sus métodos y prácticas de diseño en una sola metodología.

Los diagramas UML se dividen en dos categorías principales: diagramas estructurales y diagramas de comportamiento. Los diagramas estructurales muestran la estructura estática del sistema, mientras que los diagramas de comportamiento muestran la dinámica y las interacciones entre los elementos del sistema.

Diagramas Estructurales: Se centran en la organización y estructura del sistema.

- Diagrama de Clases: Muestra las clases del sistema, sus atributos, métodos y las relaciones entre ellas.
- Diagrama de Objetos: Representa instancias de clases en un momento específico.
- Diagrama de Componentes: Muestra los componentes físicos del sistema y sus interrelaciones.
- Diagrama de Despliegue: Representa la configuración de hardware y software del sistema.
- Diagrama de Paquetes: Agrupa elementos relacionados en conjuntos más manejables.

Diagramas de Comportamiento: Se centran en la dinámica del sistema.

- Diagrama de Casos de Uso: Describe las interacciones entre los actores externos y el sistema.
- Diagrama de Secuencia: Muestra la interacción entre objetos en una secuencia temporal.
- Diagrama de Colaboración: Representa interacciones organizadas alrededor de la estructura.

- Diagrama de Actividad: Describe el flujo de actividades y decisiones.
- Diagrama de Estados: Muestra los estados de un objeto y las transiciones entre esos estados.

Patrones de Diseño

Los patrones de diseño son soluciones probadas a problemas comunes en el diseño de software. Se clasifican en tres categorías principales:

Patrones Creacionales: Abordan la creación de objetos de manera que se adapta a las necesidades del sistema.

- Singleton: Garantiza que una clase solo tenga una instancia y proporciona un punto de acceso global a ella.
- Factory Method: Define una interfaz para crear objetos, pero permite a las subclasses alterar el tipo de objetos que se crean.
- Abstract Factory: Proporciona una interfaz para crear familias de objetos relacionados o dependientes sin especificar sus clases concretas.

Patrones Estructurales: Se ocupan de la composición de clases y objetos para formar estructuras más grandes.

- Adapter: Permite que interfases incompatibles colaboren mediante una clase que convierte la interfase de una clase en otra que el cliente espera.
- Decorator: Agrega funcionalidades adicionales a un objeto de manera dinámica.
- Facade: Proporciona una interfaz simplificada a un conjunto complejo de interfaces en un subsistema.

Patrones de Comportamiento: Se centran en la comunicación entre objetos y la asignación de responsabilidades.

- Observer: Define una dependencia uno a muchos entre objetos, de manera que cuando uno cambia de estado, todos sus dependientes son notificados y actualizados automáticamente.
- Strategy: Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables.
- Command: Encapsula una solicitud como un objeto, lo que permite parametrizar clientes con colas, solicitudes y operaciones.

3. Glosario de Terminología UML

- Clase (Class): Representa un conjunto de objetos con características y comportamientos similares. Es una unidad fundamental en la programación orientada a objetos.
- Objeto (Object): Instancia de una clase. Representa un elemento concreto en el sistema.
- Atributo (Attribute): Propiedad o característica de una clase. Define el estado de un objeto.
- Método (Method): Comportamiento o función que una clase puede realizar. Define las operaciones que un objeto puede ejecutar.
- Diagrama de Clases (Class Diagram): Diagrama estructural que muestra las clases del sistema, sus atributos, métodos y las relaciones entre ellas.
- Diagrama de Casos de Uso (Use Case Diagram): Diagrama de comportamiento que muestra las interacciones entre actores externos y el sistema para lograr un objetivo específico.
- Actor (Actor): Entidad externa que interactúa con el sistema. Puede ser una persona, un dispositivo o un sistema externo.
- Diagrama de Secuencia (Sequence Diagram): Diagrama de comportamiento que muestra cómo los objetos interactúan en una secuencia temporal para llevar a cabo un proceso.
- Diagrama de Actividad (Activity Diagram): Diagrama de comportamiento que representa el flujo de actividades y decisiones en un proceso.
- Diagrama de Estados (State Diagram): Diagrama de comportamiento que muestra los estados por los que pasa un objeto y las transiciones entre esos estados.
- Asociación (Association): Relación entre dos clases que indica cómo los objetos de esas clases pueden interactuar.
- Herencia (Inheritance): Relación entre clases donde una clase (subclase) hereda atributos y métodos de otra clase (superclase).

- Agregación (Aggregation): Tipo de asociación que representa una relación "todo/parte" donde una clase contiene otras clases, pero las partes pueden existir independientemente del todo.
- Composición (Composition): Tipo más fuerte de agregación donde las partes no pueden existir independientemente del todo.
- Interfaz (Interface): Especificación de un conjunto de métodos que una clase debe implementar. Define un contrato que las clases pueden cumplir.

4. Conclusión

Hemos definido y explicado los términos clave de las bases conceptuales acerca del lenguaje unificado de modelado (UML) y patrones de diseño.

UML es una herramienta versátil y poderosa que mejora la visualización, especificación, construcción, documentación y comunicación en el desarrollo de software. Facilita la creación de sistemas más robustos, eficientes y mantenibles, asegurando que todos los stakeholders tengan una comprensión clara y común del diseño del sistema.

5. Bibliografía

chrome-extension://efaidnbmnnnibpcajpcgiclfndmkaj/https://
zajuna.sena.edu.co/Repositorio/Titulada/institution/SENA/
Tecnologia/228118/Contenido/DocArtic/GUI4/Guia_aprendizaje_4.pdf

<https://www.lucidchart.com/pages/es/que-es-el-lenguaje-unificado-de-modelado-uml>